TAD 2026

Introducere
Situație școlară
Modulul 1 - Utilizare API-uri web
    Intro Python
    Tehnologii posibile:
    Enunț
    Exemple API-uri:
        Ex python air visual:
        Apelarea API-urilor din LabView

Introducere

Formula de calcul a notei finale este:
NF = 40% NotaTeme + 60% NotaProiect (sau referat*)
NotaTeme = (T1 + T2 + T3)/3

*nota la referat <= 7

Vor fi 3 teme, pe parcursul a primelor 9 săptămâni, planificare fiecărei teme având loc sub următoarea formă:

- S1 - prezentare
- S2 - discuții, exemplificări implementare, evaluări parțiale
- S3 - prezentarea temei, on-site, laborator E204, joia, începând cu orele 16:30-16:45, conform unei planificări afișate/trimise în timpul zilei respective

Situație școlară

… soon

Modulul 1 - Utilizare API-uri web

Intro Python

```
print('buna ziua')
a = 5
nume = 'Damian Catalin'

#a = int(input('dati un nr:'))



print(nume)

if a > 0:
    print('a este pozitiv')
    # a = a + 1
    # print(a + 1)
else:
    print('a este negativ')

for i in range(10, 50, -5):
    print( ' ', i, sep = '-', end= '\t')
```

Tehnologii posibile:

- python (Visual Studio Code) - www.infopuc.ro/python
- node.js
- .net
- labview
- java

Enunț

Utilizând API-uri publice, realizați o **interfață grafică** sau text care va prelua datele de la cel puțin două API-uri și va afișa rezultate relevante (de natură de mediu, sau altă natură).
Vor obține punctaj maxim temele care:

- vor folosi datele returnate de anumite API pentru interogarea altor API-uri

- vor prezenta datele sub formă grafică/window
- vor permite și alte customizări în cererea informațiilor (de exemplu, se pot solicita automat datele pentru locația curentă și la cerere, informații despre alte orașe/țări, etc.)
- **BONUS:** se vor interoga API-uri diferite, urmând ca anumite informații obținute de la acestea să fie folosite pentru interogarea altor API-uri.
- Termen de predare (12 Martie 2026). Se pot prezenta și ulterior, dar vor suferi mici penalizări.

Exemple API-uri:

- https://www.airvisual.com/ (https://www.iqair.com/dashboard/api)
- https://openweathermap.org/
- https://rapidapi.com/blog/ip-geolocation-api
- https://opencagedata.com/api
- https://flickr.com
- https://apilayer.com/#products
- Alexandru Glodeanu - 2021
  - https://rapidapi.com/wirefreethought/api/geodb-cities
  - https://www.climacell.co/weather-api
- Daniel Bejenar - 2021
  - https://fixer.io/product
- Paul Crețu - 2021
  - https://xyz.api.here.com/hub/static/swagger/
  - https://developer.paypal.com/docs/integration/direct/payments/
  - Travel Booking
- Iustin Boureanu - 2021
  - https://any-api.com/
- Dragoș Crîșmaru - 2021
  - https://ipstack.com/
- Vlad Corduneanu - 2021
  - https://api.nasa.gov/
- Mercic Cristian - 2024
  - https://unsplash.api

Ex python air visual:

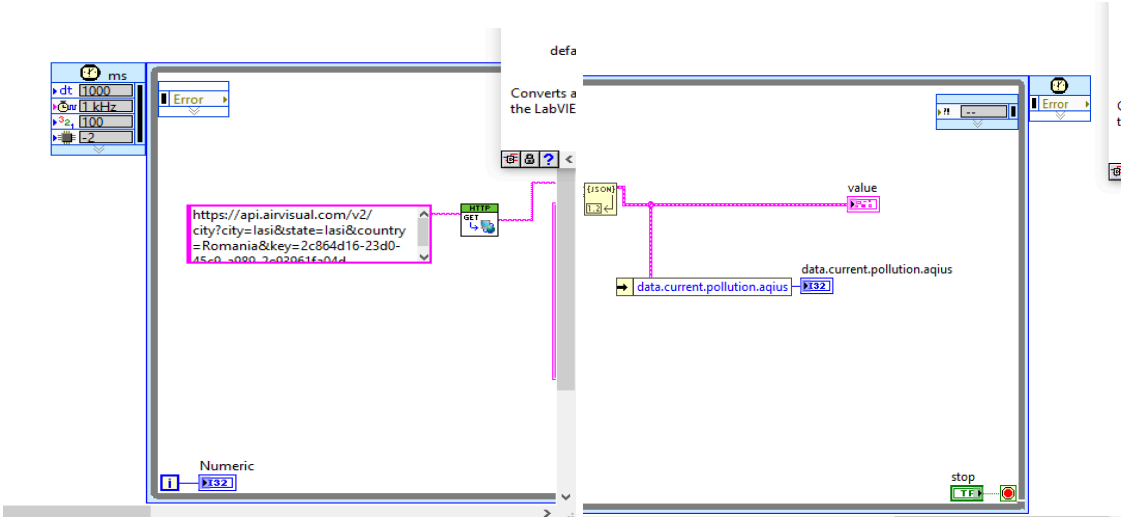
test.py

```
import requests
```

```
import json

url                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                =
"https://api.airvisual.com/v2/city?city=Iasi&state=Iasi&country=Romania&key=cheia_personala"


payload = {}
headers= {}

response = requests.request("GET", url, headers=headers, data = payload)

print(response.text.encode('utf8'))

obj = json.loads(response.text.encode('utf8'))
print(obj['data']['location']['coordinates'])
print(obj['data']['country'],obj['data']['city'],
obj['data']['current']['pollution']['aqius'])
```

```
$: Python3 test.py
```

Apelarea API-urilor din LabView



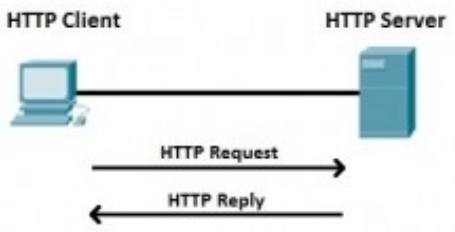
Modulul 2 - Realizarea unui API

Introducere

În etapa precedentă am văzut ce reprezintă un API și cum sunt accesate resursele existente în cadrul API-urilor publice. În cadrul prezentului subiect vom încerca realizarea unui API astfel încât să expunem niște informații (care pot fi și date de măsurare).

Server http

HTTP: Hypertext Transfer Protocol. Reprezintă protocolul utilizat de www pentru transmiterea și reprezentarea informațiilor. Este un set de reguli pentru transferul fișierelor (text, grafica, sunet, video, etc). Funcționează pe bază de cerere-răspuns, client-server. Clientul trimite o cerere http către server, iar serverul răspunde cu informații, acțiuni sau alte activități în numele clientului și a cererii acestuia.



Request

Un mesaj simplu de solicitare de la un client este format din următoarele componente:

- o linie de solicitare pentru a obține o resursă necesară, de exemplu, o solicitare GET /content/page1.html solicită de la server o resursă numită /content/page1.html
- Anteturi (Exemplu - Accept-Language: EN)
- O linie goală (fără spații)
- Un corp de mesaje care este opțional.

Toate liniile trebuie să se încheie cu un caracter de linie nouă.

Response

Un răspuns simplu din partea serverului conține următoarele componente:

- Cod de stare HTTP (De exemplu, HTTP / 1.1 301 Mutat permanent, înseamnă că resursa solicitată a fost mutată permanent și clientul este redirecționat către o altă resursă)
- Anteturi (Exemplu - Tip conținut: html)
- O linie goală
- Un corp de mesaje care este opțional.

Toate liniile din răspunsul serverului ar trebui să se încheie cu enter.

Python

Codul sursă pentru un server http este prezentat mai jos:
https://www.youtube.com/watch?v=mqhxxeeTbu0

```
from flask import Flask, redirect, url_for

app = Flask(__name__)

@app.route("/")
def home():
      return "Hello. The main page. <h1>Hello</h1>"

@app.route("/<test>")
def user(test):
      return f"Hello {test}!"

@app.route("/admin")
def admin():
      return redirect(url_for("home"))

if __name__ == "__main__":
      app.run()
```

Pentru rularea codului de mai sus se scrie în consolă:

```
python3 nume_fisier.py
```

În consolă se va afișa portul la care serverul "așteaptă" apelurile.

Node.js

Codul sursă pentru un server http este prezentat mai jos:
https://nodejs.org/en/knowledge/HTTP/servers/how-to-create-a-HTTP-server/

```
const http = require('http');

const requestListener = function (req, res) {
  res.writeHead(200);
```

```
  res.end('Hello, World!');
}

const server = http.createServer(requestListener);
server.listen(8080);
```

Pentru rularea codului de mai sus se scrie în consolă:

```
node nume_fisier.js
```

În consolă se va afișa statusul serverului și portul (8080) la care serverul ”așteaptă” requesturile.

Protocol *stateless*

HTTP se numește protocol *stateless*, deoarece fiecare comandă este executată independent, fără a se cunoaște comenzile anterioare. Serverul primește solicitarea, o procesează și creează un răspuns pentru client. Odată ce răspunsul este transferat clientului, conexiunea este încheiată și următoarea solicitare de la același client este tratată independent.

Pentru a modela un API va fi nevoie inițial de un server http ***up & running***. Realizarea unui server http se poate face folosind un limbaj de programare uzual (Python, Node.js, Java, etc.).

API

Modelare API

Pentru realizarea unui API, înainte de partea de programare efectivă este necesară o modelarea a ”comportamentului” acesteia. Putem specifica, pe scurt, că cererile către serverele/paginile web sunt alcătuite din: **verbe (metode)**, **conținut (body)** și au ca efect un **răspuns**.

Verbe http

Verbele utilizate în cadrul comunicării client-server (doar cele mai importante) sunt:

- **GET**: solicită o reprezentare a resursei specificate. Solicitările care utilizează GET ar trebui să solicite date și nu ar trebui să aibă niciun alt efect.
- **POST**: solicită ca serverul să accepte entitatea inclusă în cerere ca o nouă subordonată a resursei web identificate de URI (Uniform Resource Identifier). Datele publicate ar putea fi, de exemplu, o adnotare pentru resursele existente; un mesaj pentru un forum, un grup de știri, o listă de corespondență sau un subiect de comentarii; un bloc de date care este rezultatul trimiterii unui formular web la un proces de gestionare a datelor; sau un element de adăugat la o bază de date.

- **PUT**: solicită ca entitatea închisă să fie stocată sub URI-ul furnizat. Dacă URI se referă la o resursă deja existentă, aceasta este modificată; dacă URI nu indică o resursă existentă, atunci serverul poate crea resursa cu acel URI.
- **DELETE**: șterge resursa specificată.
- **HEAD**: solicită un răspuns identic cu cel al unei solicitări GET, dar fără corpul de răspuns. Acest lucru este util pentru preluarea meta-informațiilor scrise în anteturile de răspuns, fără a fi necesar transferul întregului conținut.
- **TRACE**: este un ecou la solicitarea primită, astfel încât un client să poată vedea ce (dacă există) modificări sau adăugiri au fost făcute de servere intermediare.
- **OPTIONS**: returnează metodele HTTP pe care serverul le acceptă pentru adresa URL specificată. Aceasta poate fi utilizată pentru a verifica funcționalitatea unui server web solicitând „*” în loc de o resursă specifică.
  Toate serverele HTTP cu scop general implementează cel puțin metodele GET și HEAD, celelalte metode fiind considerate opționale prin specificație.
  Unele dintre metodele (de exemplu: HEAD, GET, OPTIONS și TRACE) sunt, prin convenție, definite ca sigure, ceea ce înseamnă că sunt destinate doar preluării informațiilor și nu ar trebui să schimbe starea serverului.

Coduri de stare

În funcție de disponibilitatea Resursei, HTTP furnizează un cod de stare adecvat (denumit și „Răspuns server”) determinat de protocol. Acestea sunt următoarele:

- 1xx - doar un mesaj informativ
- **2xx - succes, de exemplu: 200 OK - fișier găsit**
- 3xx - redirecționează clientul către o altă adresă URL, de exemplu: 301 s-a mutat permanent
- **4xx - eroare din partea clientului, de exemplu: 404 - fișierul nu a fost găsit**
- 5xx - eroare din partea serverului, de exemplu: 500 eroare server

API RESTfull

https://en.wikipedia.org/wiki/Representational_state_transfer

HTTP METHODS	Collection resource, such as `http://ip/masini`	Member resource, such as `http://ip/masini/masina_1`
GET	*Retrieve* the URIs of the member resources of the collection resource in the response body.	*Retrieve* representation of the member resource in the response body.

	https://api.example.com/masini/masina_1
	
	https://api.example.com/masini/masina_2
	
	https://api.example.com/masini/masina_3
	
	Poate fi si o lista a item-urilor
		{
		
		"id" : 1,
		
		"model": 2008,
		
		"brand": "Audi"
		
		}

POST	***Create* a member resource in the collection** resource using the instructions in the request body. The URI of the created member resource is *automatically assigned* and returned in the response *Location* header field.	*Create* a member resource in the member resource using the instructions in the request body. The URI of the created member resource is *automatically assigned* and returned in the response *Location* header field.
		
		{
		
		"id":"cdamian",
		
		"pass":"******",
		
		"email":{
		
		"email1":"cdamian@tuiasi.ro",
		
		"email2:"catalin.damian@academic.....ro"
		
		},
		
		"title":"fgh"
		
		}

PUT	*Replace* all the representations of the member resources of the collection resource with the representation in the request body, or *create* the collection resource if it does not exist.	*Replace* all the representations of the member resource or *create* the member resource if it does not exist, with the representation in the request body.
PATCH	*Update* all the representations of the member resources of the collection resource using the instructions in the request body, or *may create* the collection resource if it does not exist.	*Update* all the representations of the member resource, or *may create* the member resource if it does not exist, using the instructions in the request body.

DELETE	*Delete* all the representations of the member resources of the collection resource.	*Delete* all the representations of the member resource.

Enunț temă

Să se modeleze și să se implementeze un API RESTfull care să expună informații referitoare la măsurători în orice domeniu (mediu, energie, electronică, etc).

- API-ul va expune informații structurate JSON și va avea implementate cel puțin verbele **GET, POST, PUT și DELETE**, atât pentru **colecția de date** cât și pentru **fiecare element** în parte
- Răspunsurile serverului vor fi obligatoriu însoțite de coduri de stare corespunzătoare (200, 201, 400, 404, etc)
- Prezentare temă: 2 Aprilie. Planificarea pentru prezentarea temei va avea în vedere doar formularele (și sursele temelor) trimise până în ziua predării, până la ora 10.
- atenție, tema trebuie sa aibă o aplicabilitate logică, reală (de ex. nu se vor afișa 3.45 mașini)
- Vor primi bonus temele care:
    - sunt realizate in LabView.
    - au implementată și partea de client web pentru testarea aplicației.

Testarea API

Collection

http://localhost:3000/fruits

GET

Preia lista

Request

Method
GET

Request URL
http://localhost:3000/fruits

SEND

Parameters

Headers | Variables

Toggle source mode Insert headers set

Header name: authorization
Header value: Bearer a0dc35cf6295cca3133409145b5d358675c3d2a5

ADD HEADER

Headers are valid Headers size: 63 bytes

200 OK 15.99 ms DETAILS


```
[Array[4]
  -0: {
        "id": 1,
        "name": "Portocale",
        "price": 200
      },
  -1: {
        "id": 2,
        "name": "Banane",
        "price": 231
      },
  -2: {
        "id": 3,
        "name": "Capsuni",
        "price": 250
      },
  -3: {
        "id": 4,
        "name": "Cirese",
        "price": 100
      }
]
```


POST

Adaugă un item

Body: **{"id":6,"name":"mere","price":10}**

TAD 2026

Body content type | Raw input


```
{"id":6,"name":"mere","price":10}
```


201 Created 26.00 ms DETAILS

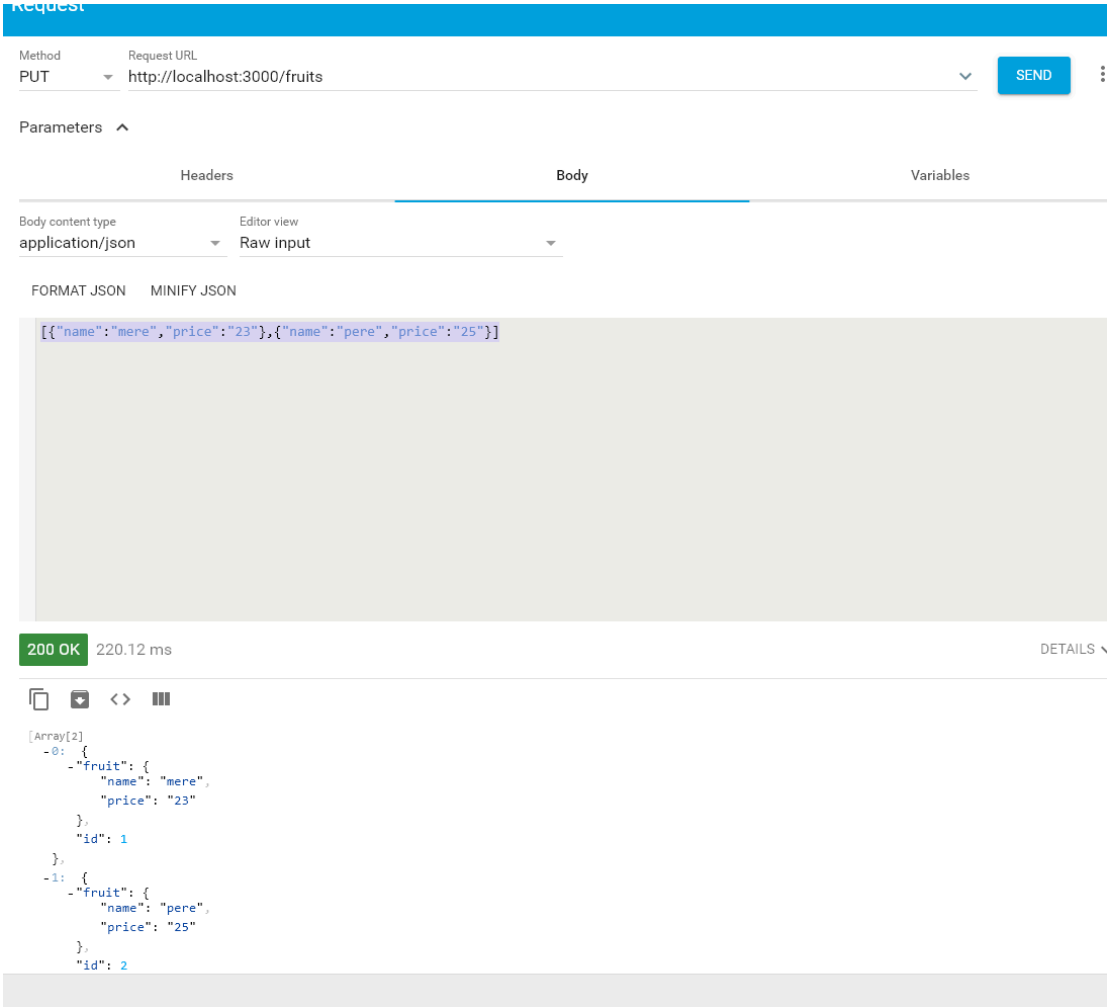

```
[Array[5]
 -0: {
      "id": 1,
      "name": "Portocale",
      "price": 200
  },
 -1: {
      "id": 2,
      "name": "Banane",
      "price": 231
  },
 -2: {
      "id": 3,
      "name": "Capsuni",
      "price": 250
  },
 -3: {
      "id": 4,
      "name": "Cirese",
      "price": 100
  },
 -4: {
      "id": 6,
      "name": "mere",
      "price": 10
  }
],
```

TAD 2026

PUT

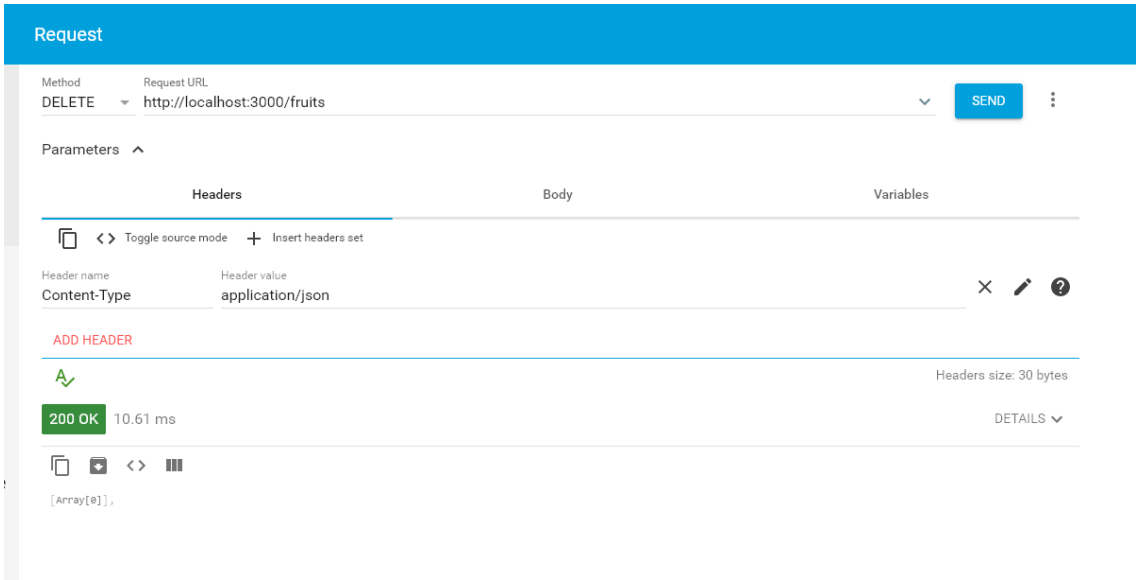
- adaugă/înlocuiește lista

[{"name":"mere","price":"23"},{"name":"pere","price":"25"}]



DELETE

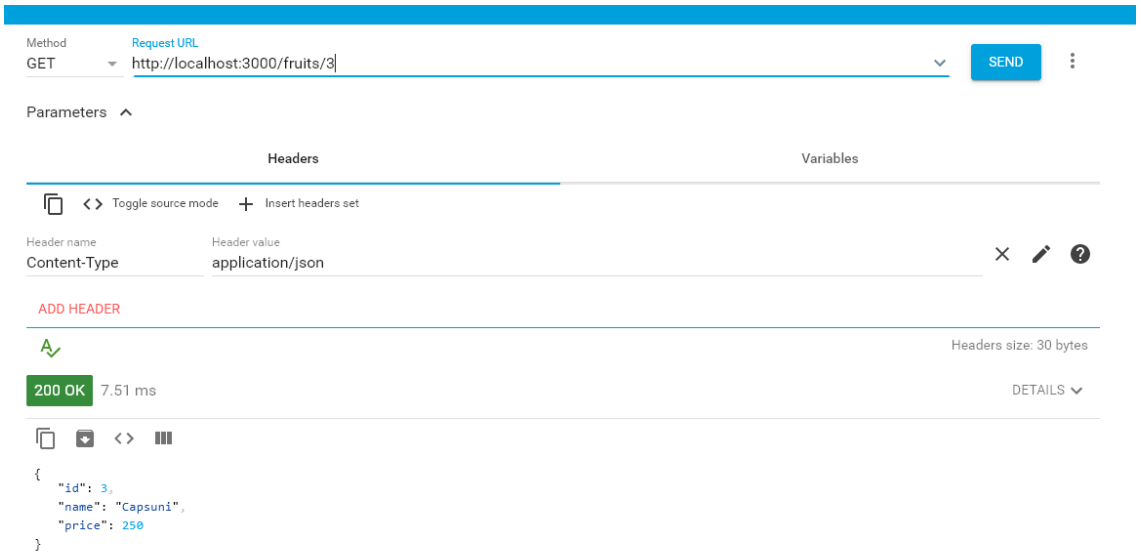
- Șterge lista de item-uri



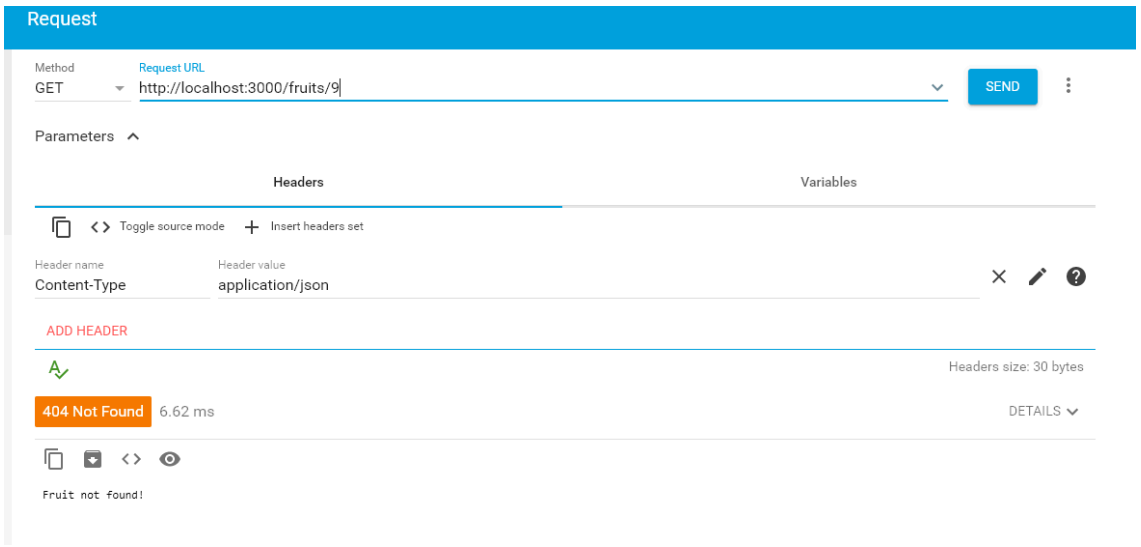
Item

GET

Caz OK: http://localhost:3000/fruits/3



Caz greșit: http://localhost:3000/fruits/9



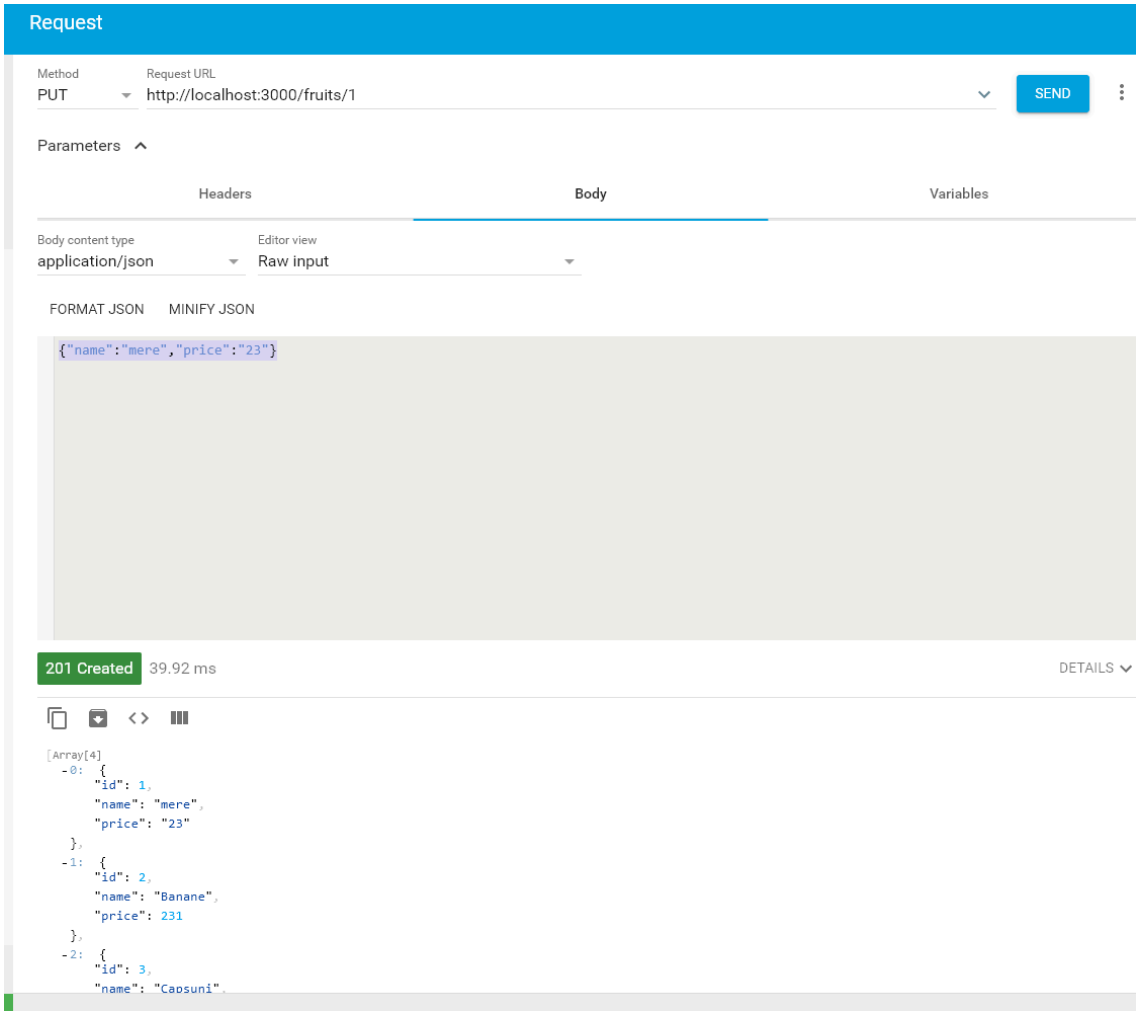
POST

- Ne-exemplificat

PUT

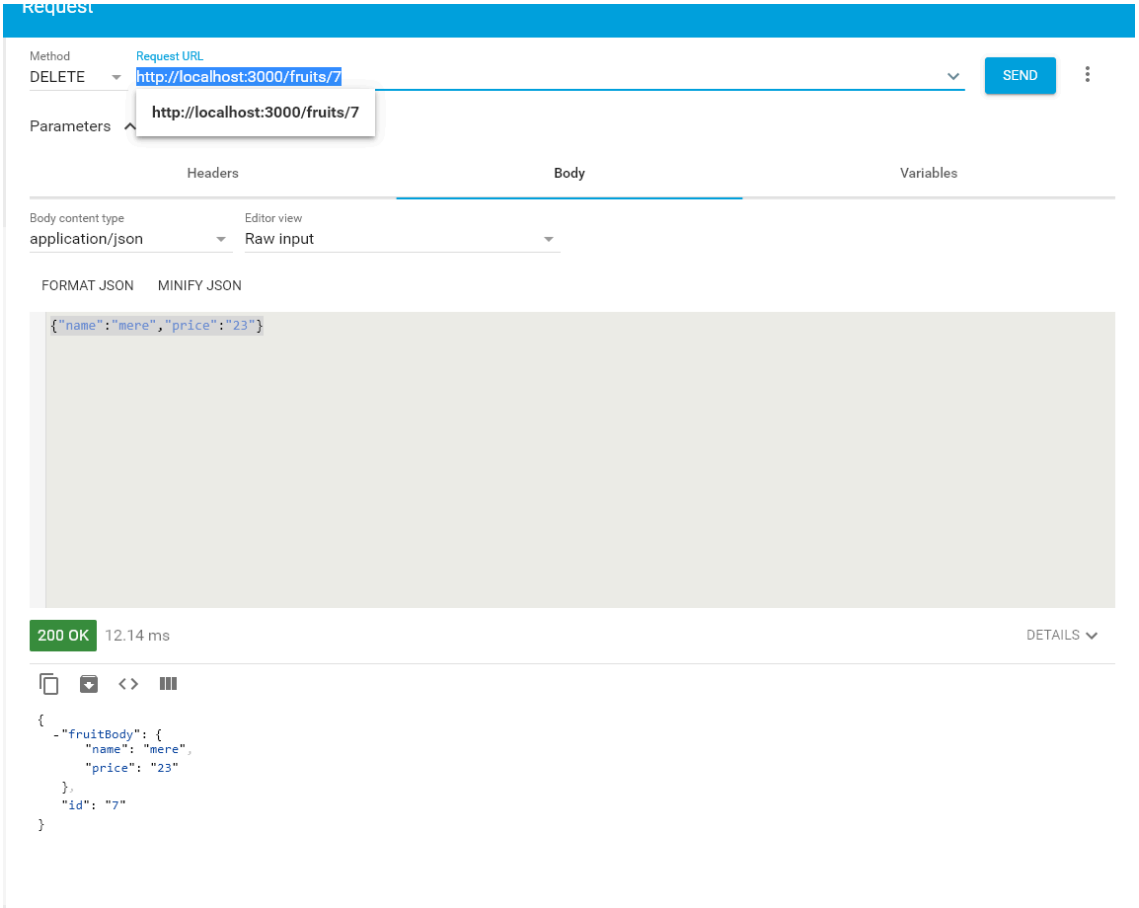
http://localhost:3000/fruits/1
{"name":"mere","price":"23"}

- Dacă resursa există: update item
- Dacă resursa nu există: add item



DELETE

http://localhost:3000/fruits/7



Tutorial creare servicii web în LabVIEW:

https://zone.ni.com/reference/en-XX/help/371361R-01/lvhowto/build_web_service/

Alte resurse:

Clienti HTTP

- REST CLIENT (https://install.advancedrestclient.com/)

TAD 2026

- POSTMAN

https://www.youtube.com/watch?v=GMppyAPbLYk

- Despre REST
- Dileme? -> GET,POST, HEAD, PUT, DELETE
- Implementing REST
- SOAP vs REST
- REST Protocol;
- POST,PUT si idempotenta
- Create A REST API With JSON Server
- HTTP requests in Java Script
- Node.js Middleware to Log HTTP API Requests and Responses
- RoyFielding - about REST API
- ngrok - Expose a local web server to the internet
- REST API Tutorial